“Image Segmentation of Multiple Coin Objects Using” MATLAB

Project Report

COURSE

Digital Image Processing

SUBMITTED BY:

Ali Hamza (230666)

Muhammad Zohaib (230602)

Bachelor of Computer Engineering 2023-27

SUBMITTED TO:

Miss Zoafshan

Department of Electrical and Computer Engineering

AIR UNIVERSITY ISLAMABAD

Table of Contents

Abstract

Image segmentation is a fundamental technique in Digital Image Processing that is used to identify and separate objects of interest within an image. In this project, a dataset of coin images was processed using MATLAB to perform segmentation and object counting. Different image processing techniques, including grayscale conversion, noise reduction, contrast enhancement, thresholding, Sobel edge detection, and watershed segmentation, were implemented to extract coin regions from the background. The segmented images were further analyzed to determine the number of coins present in each image. The results obtained demonstrate that classical image processing techniques can effectively perform object segmentation and counting with satisfactory accuracy.

Introduction

Image segmentation is the process of partitioning an image into multiple meaningful regions to simplify image analysis and interpretation. It is one of the most important operations in Digital Image Processing and is widely used in applications such as object recognition, medical imaging, industrial inspection, and computer vision systems.

The purpose of this project is to apply image segmentation techniques to a dataset of coin images using MATLAB. The selected images contain multiple coins placed on a background, with some images including coins that are touching or partially overlapping. The project focuses on separating individual coins from the background, detecting object boundaries, and accurately counting the number of coins present in each image. By combining different segmentation techniques, reliable object extraction and analysis can be achieved.

Objectives

- To perform image preprocessing on coin images using MATLAB.
- To improve image quality through filtering and contrast enhancement techniques.
- To separate coin objects from the background using thresholding.
- To identify object boundaries using the Sobel edge detection method.
- To separate touching coins using watershed segmentation.

Dataset Description

The dataset consists of multiple coin images collected from publicly available sources. Each image contains several coins placed on a relatively uniform background. Some images contain well-separated coins, while others include touching or partially overlapping coins, making the segmentation process more challenging. All images were resized to a fixed resolution before processing to ensure consistency throughout the experiment.

Methodology

1 Image Acquisition

The input coin images were loaded into MATLAB and resized to a resolution of 400 × 600 pixels. Resizing the images ensured uniformity and facilitated consistent processing across the entire dataset.

CODE:

/MATLAB Drive/DIP_CEP/DIP_CEP/CEP_DIP_1.m

```
1   Z = {'coin1.jpg','coin2.jpg','coin3.jpg','coin4.jpg','coin5.jpg','coin6.jpg','coin7.jpg','coin8.jpg'};
2
3   for k = 1:length(Z)
4
5   img = imread(Z{k});
6   img = imresize(img,[400 600]);
7
```

2 Grayscale Conversion

The acquired RGB images were converted into grayscale format using the MATLAB **rgb2gray** function. Since segmentation primarily relies on intensity information, grayscale conversion reduces computational complexity while preserving essential image features.

CODE:

```
8   gray=rgb2gray(img);
```

3 Noise Removal

To reduce unwanted noise and smooth intensity variations, a 3 × 3 averaging filter was applied to each image. This preprocessing step improves image quality and enhances the effectiveness of subsequent segmentation operations.

CODE:

```
kernel = ones(3,3)/9;

filtered = conv2(double(gray),kernel,'same');
filtered = uint8(filtered);

```

4 Contrast Enhancement

Image contrast was improved through intensity normalization. The pixel values were stretched between the minimum and maximum intensity levels, resulting in better visibility of object boundaries and improved segmentation performance.

CODE:

```

minVal = double(min(filtered(:)));
maxVal = double(max(filtered(:)));

enhanced = uint8(255 * ((double(filtered)-minVal) / (maxVal-minVal)));

```

5 Thresholding

Automatic threshold selection was performed using Otsu's thresholding method. This technique converts the grayscale image into a binary image by separating foreground objects from the background based on intensity distribution.

CODE:

```
histCounts = zeros(1,256);%Thresholding

for p = 0:255
    histCounts(p+1) = sum(sum(enhanced == p));
```

```
end

totalPixels = numel(enhanced);

prob = histCounts / totalPixels;

bestThreshold = 0;
bestVariance = 0;

for t = 1:254

    w0 = sum(prob(1:t));
    w1 = sum(prob(t+1:end));

    if w0 == 0 || w1 == 0
        continue;
    end

    mu0 = sum((0:t-1) .* prob(1:t)) / w0;
    mu1 = sum((t:255) .* prob(t+1:end)) / w1;

    betweenVar = w0 * w1 * (mu0 - mu1)^2;

    if betweenVar > bestVariance
        bestVariance = betweenVar;
        bestThreshold = t;
    end

end

binary1 = double(enhanced > bestThreshold);
binary2 = double(enhanced < bestThreshold);

count1 = sum(binary1(:));
count2 = sum(binary2(:));

if count1 < count2
    binaryImage = binary1;
else
    binaryImage = binary2;
end
```

---

6 Binary Image Cleaning

Several morphological operations were performed to refine the binary image and eliminate segmentation errors. These operations included:

• Removal of small noisy regions.

• Filling holes present inside segmented coin regions.

• Elimination of objects connected to image boundaries.

These steps significantly improved the quality of the segmented output.

CODE:

```
64
65                                          % Binary Image Cleaning
66      binaryImage = bwareaopen(binaryImage,500);
67
68      binaryImage = imfill(binaryImage,'holes');
69
70      binaryImage = imclearborder(binaryImage);
71
```

7 Sobel Edge Detection

The Sobel edge detection operator was applied to identify object boundaries. Horizontal and vertical image gradients were calculated and combined to obtain the edge magnitude, resulting in clear visualization of coin outlines.

CODE:

```
                                                    % Sobel Edge Detection
smoothed = uint8(conv2(double(enhanced),ones(5,5)/25,'same'));

sobelX = [-1 0 1; -2 0 2; -1 0 1];
sobelY = [-1 -2 -1; 0 0 0; 1 2 1];

Gx = conv2(double(smoothed),sobelX,'same');
Gy = conv2(double(smoothed),sobelY,'same');

edgeMagnitude = sqrt(Gx.^2 + Gy.^2);

edges = edgeMagnitude > 180;

```

8 Watershed Segmentation

Watershed segmentation was employed to separate touching or overlapping coins. The method utilizes distance transformation and region-based analysis to identify individual objects within connected regions. This approach proved effective in distinguishing adjacent coins that could not be separated through thresholding alone.

CODE:

```
                                                    % Watershed Segmentation
D = bwdist(~binaryImage);

D = -D;

D = imhmin(D,2);

L = watershed(D);

L(~binaryImage) = 0;

watershedRGB = label2rgb(L,@jet,'k','shuffle');

```

9 Coin Counting

Connected component analysis was performed on the segmented image to identify individual objects. Regions having an area greater than a predefined threshold were considered valid coin objects. The total number of detected coins was then calculated and displayed for each image.

CODE:

```
                                                    % Coin Counting
cc = bwconncomp(binaryImage);

stats = regionprops(cc,'Area');

areas = [stats.Area];

numCoins = sum(areas >5000);

```

Results and Discussion

The implemented system successfully segmented coin objects from the background and identified individual regions with good accuracy. Thresholding effectively separated foreground objects, while Sobel edge detection highlighted the boundaries of the coins. Watershed segmentation further improved the results by separating touching coins into distinct regions.

The experimental results indicate that the combination of preprocessing, thresholding, edge detection, and watershed segmentation provides reliable performance for coin segmentation and counting. The detected number of coins closely matched the actual number of coins present in the images, demonstrating the effectiveness of the proposed approach.

Display Section

CODE:

```
                                                    % Display Results
figure('Name',['Image ' num2str(k)],'NumberTitle','off');

subplot(2,3,1);
imshow(img);
title('Original Image');

subplot(2,3,2);
imshow(gray);
title('Grayscale Image');

subplot(2,3,3);
imshow(enhanced);
title('Preprocessed Image');

subplot(2,3,4);
imshow(binaryImage);
title(['Threshold Result T = ' num2str(bestThreshold)]);
```

Cont…

```
subplot(2,3,5);
imshow(edges);
title('Sobel Edge Detection');

subplot(2,3,6);
imshow(watershedRGB);
title(['Coins = ' num2str(numCoins)]);

sgtitle(['Processing : ' Z{k}]);

end
```

RESULTED IMAGES:

IMAGE 1:

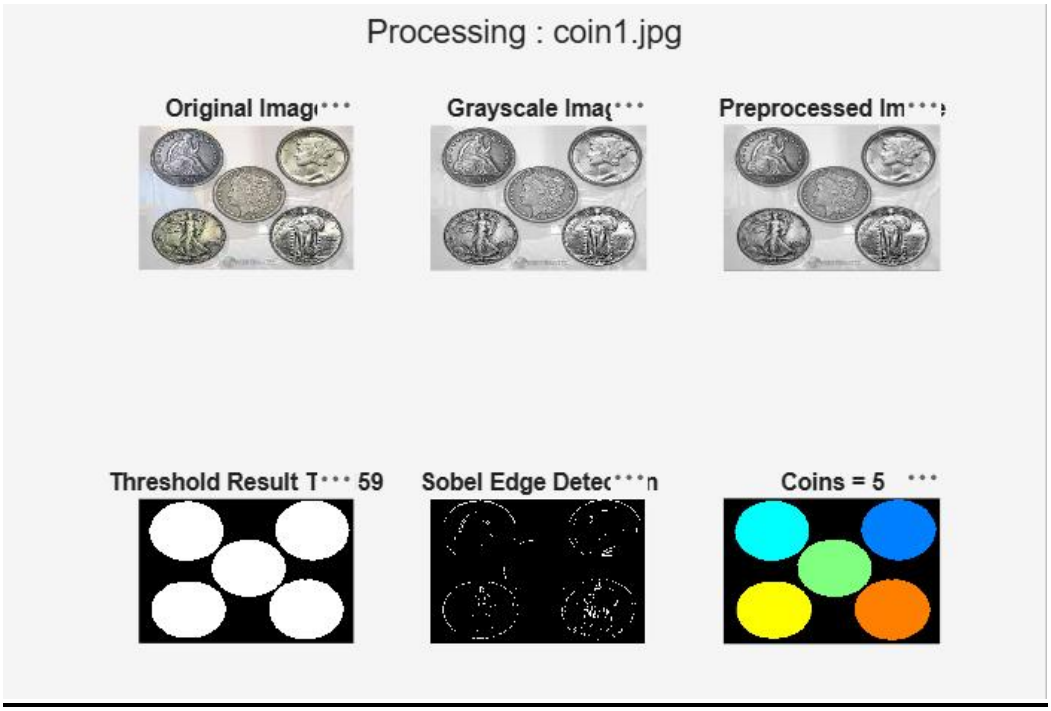


IMAGE 2:

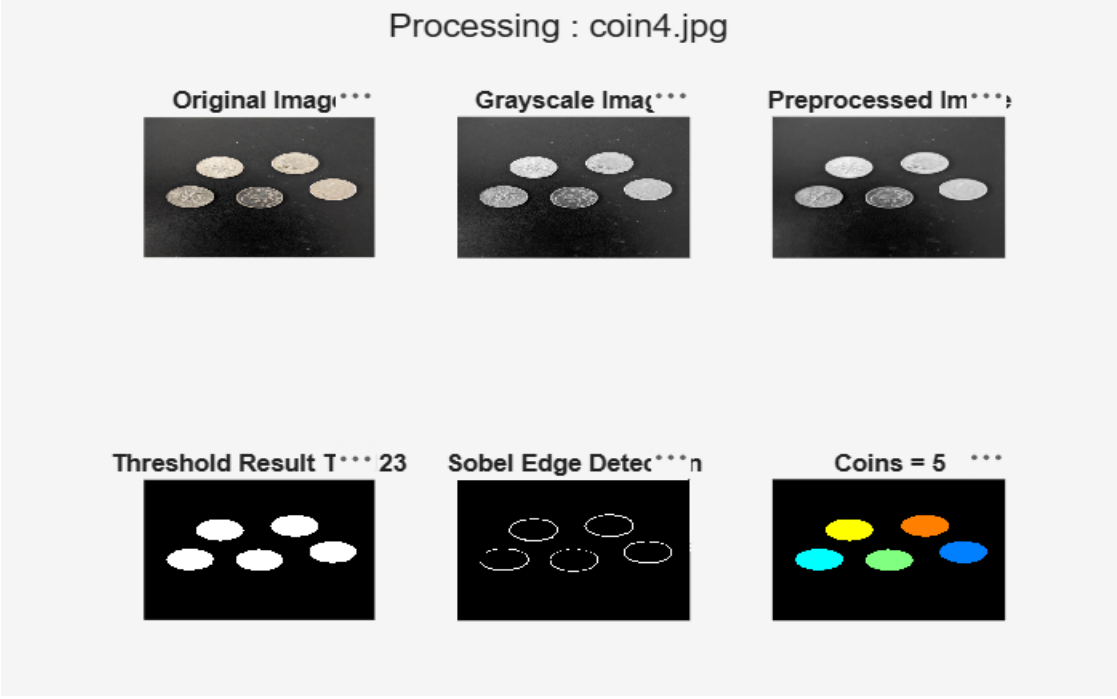


IMAGE 3:

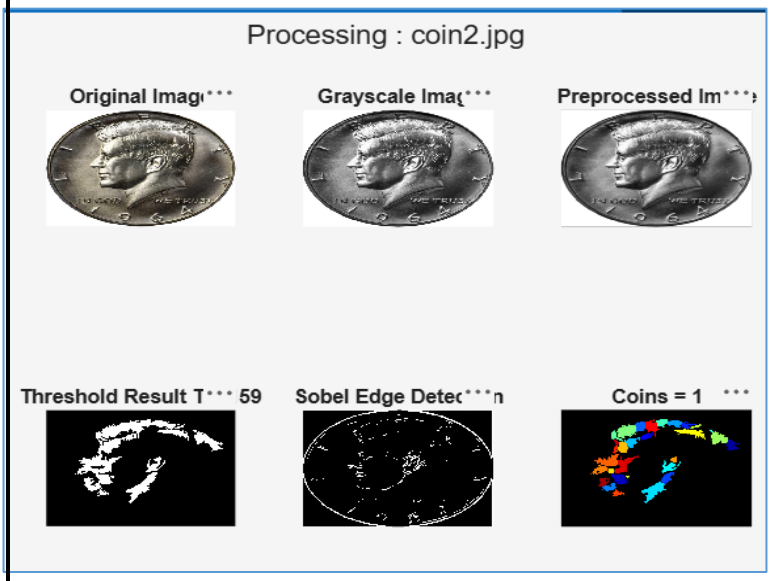


IMAGE 4:

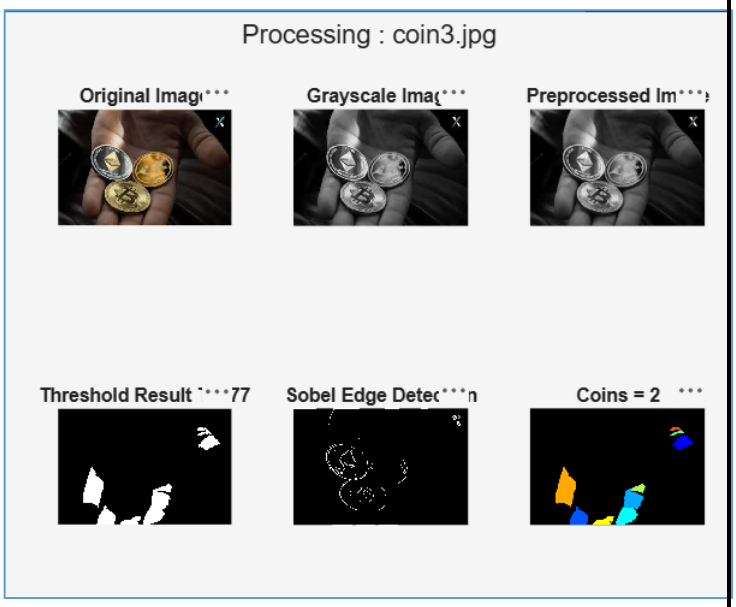


IMAGE 5:

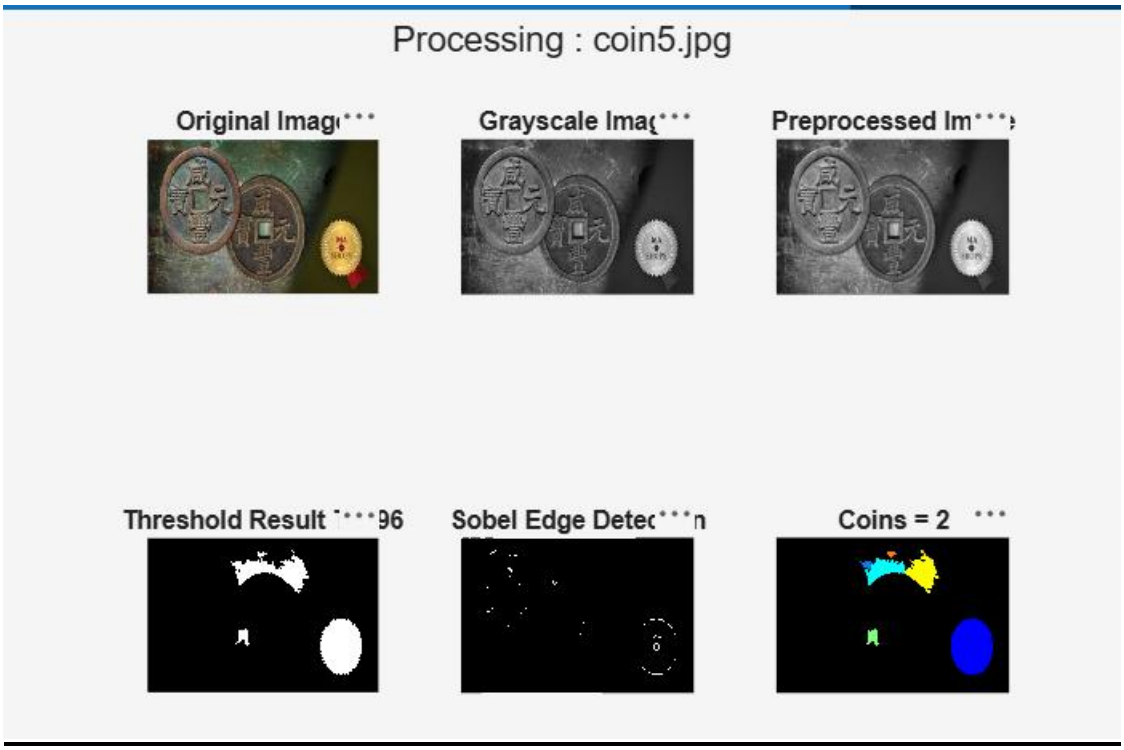


IMAGE 6:

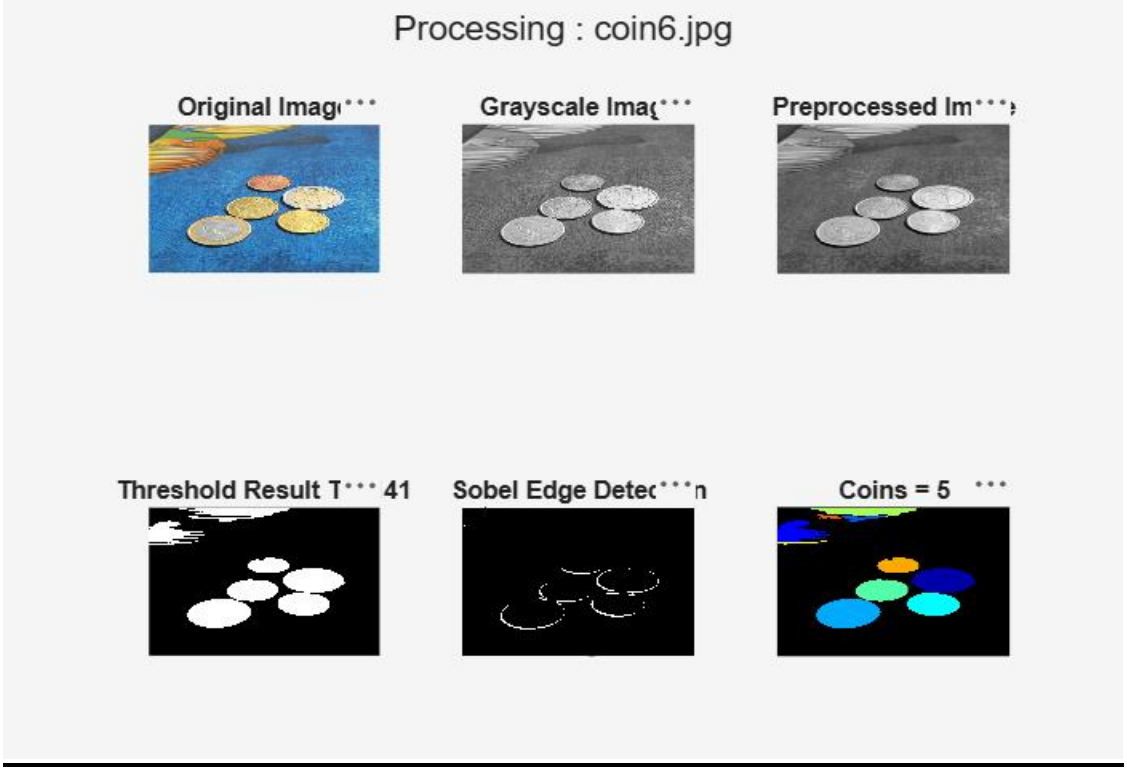


IMAGE 7:

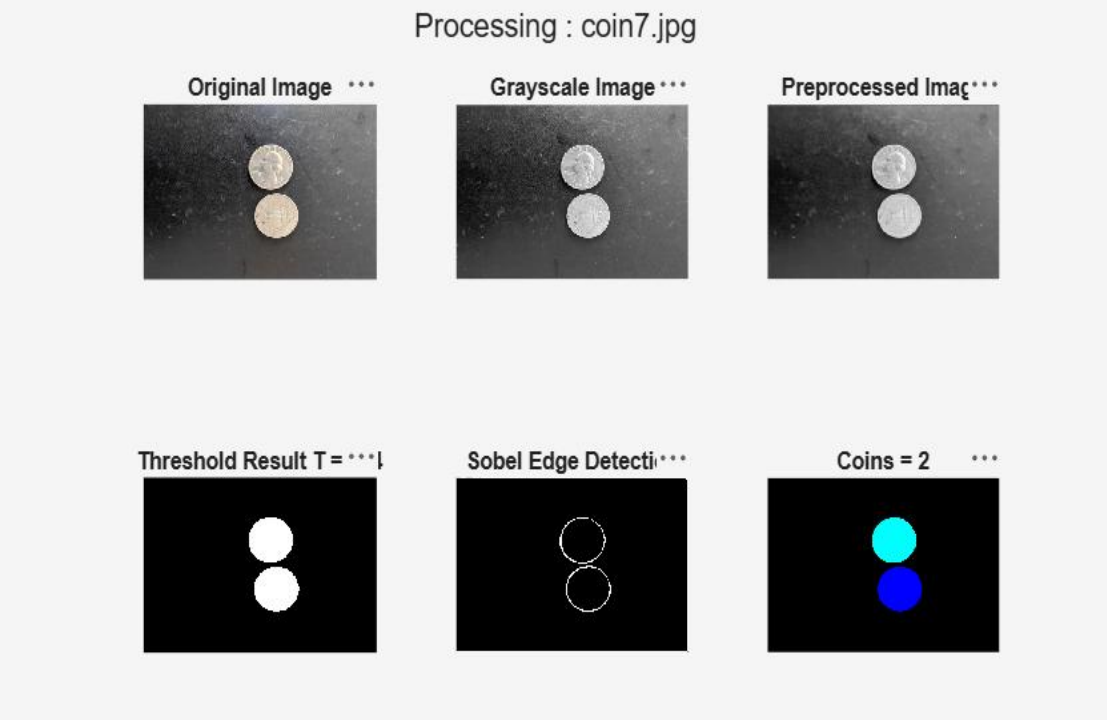
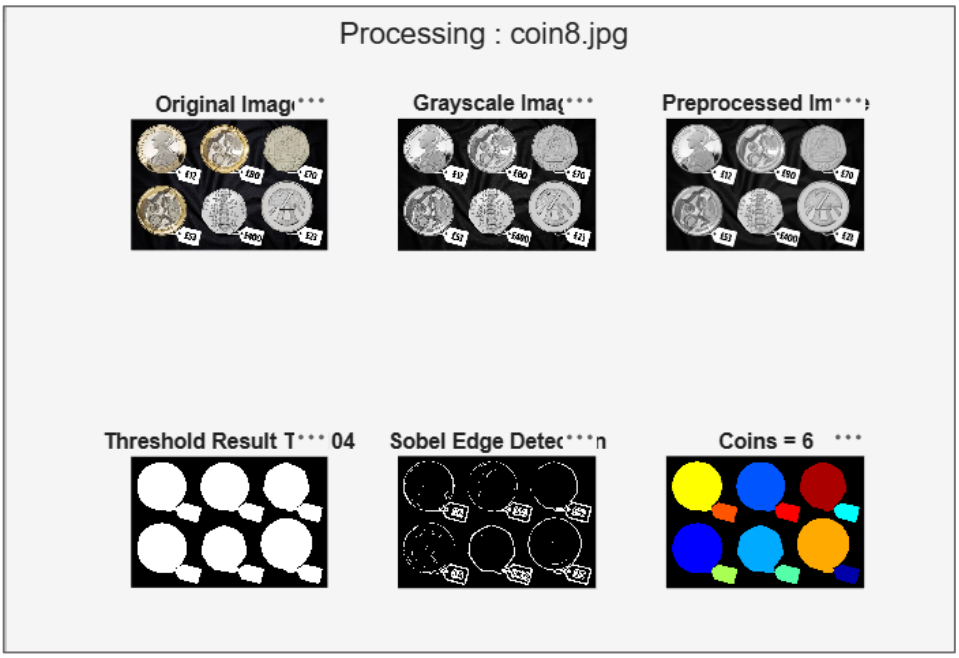


IMAGE 8:



Advantages

• Simple and straightforward implementation.

• Low computational requirements.

• Accurate detection and counting of coin objects.

• Effective separation of touching coins.

• Suitable for educational and research-based applications.

Limitations

• Performance may be affected by poor image quality.

• Uneven illumination and strong shadows can influence threshold selection.

• Highly overlapping objects may reduce segmentation accuracy.

• Parameter values may require adjustment for different datasets.

Future Work

The performance of the proposed system can be further improved by incorporating machine learning and deep learning techniques for object segmentation. Future work may also include automatic classification of different coin types and evaluation of the system on more complex datasets containing varying backgrounds and lighting conditions.

Conclusion

This project presented the implementation of classical image segmentation techniques for coin detection and counting using MATLAB. A sequence of preprocessing, thresholding, edge detection, and watershed segmentation operations was applied to accurately separate coin objects from the background. The obtained results demonstrate that traditional image processing methods can provide reliable segmentation performance while maintaining low computational complexity. The project also provided practical experience in applying Digital Image Processing concepts to a real-world object analysis problem.

References:

BOOK:

Digital Image Processing -2nd-Edition

TOOLS Used:

MATLAB 2023a

DATASET:

https://www.kaggle.com/datasets/balabaskar/count-coins-image-dataset